

Splunk – Methoden der Anomalieerkennung für Metriken

1. Native SPL-Funktionen (ohne MLTK)

`anomalydetection`-Befehl

Der einfachste Einstieg – direkt in SPL nutzbar:

```
spl
| mstats avg(cpu_usage) WHERE index=metrics span=1m
| anomalydetection cpu_usage
```

Splunk berechnet intern einen **Interquartilsabstand (IQR)**-basierten Score. Jeder Datenpunkt bekommt ein `anomaly`-Flag (0/1) und einen `anomalyScore`. Kein Training nötig, keine externe Abhängigkeit – aber rein univariat und ohne Saisonalitätskorrektur.

`trendline` + manuelle Abweichungsberechnung

Für Trend-basierte Anomalieerkennung:

```
spl
| mstats avg(latency_ms) AS latency WHERE index=metrics span=5m
| trendline sma20(latency) AS trend
| eval deviation = abs(latency - trend)
| eval anomaly = if(deviation > 2 * stdev(latency), 1, 0)
```

`streamstats` – rollierender Z-Score

Univariate Echtzeit-Anomalieerkennung ohne festes Trainingsset:

```
spl
| mstats avg(error_rate) AS error_rate WHERE index=metrics span=1m
| streamstats window=60 mean(error_rate) AS rolling_mean
                        stdev(error_rate) AS rolling_std
| eval z_score = abs((error_rate - rolling_mean) / rolling_std)
| eval anomaly = if(z_score > 3, 1, 0)
| where anomaly = 1
```

Das Fenster (`window=60`) entspricht dem Lookback-Zeitraum. Bei `span=1m` und `window=60` entspricht das einer rollierenden Stunde.

`predict`-Befehl (Zeitreihenprognose)

Splunk bringt einen eingebauten **Holt-Winters-Algorithmus** für saisonale Zeitreihen:

```
spl

| mstats avg(throughput) AS throughput WHERE index=metrics span=5m
| predict throughput
  algorithm=LLT
  future_timespan=12
  upper95=upper
  lower95=lower
| eval anomaly = if(throughput > upper OR throughput < lower, 1, 0)
```

Algorithmen im `predict`-Befehl:

Algorithmus	Beschreibung
<code>LLP</code>	Lokales Level + Saisonalität (additiv)
<code>LLT</code>	Lokales Level + Trend (kein Seasonal)
<code>LLP5</code>	LLP mit 5-Perioden-Saisonalität
<code>LL</code>	Nur lokales Level (einfachstes Modell)

```
spl

| predict throughput
  algorithm=LLP
  period=288          # 288 * 5min = 24h Saisonalität
  holdback=288        # Letzte 24h als Testset zurückhalten
  upper95=upper
  lower95=lower
```

`eventstats` / `stats` - Gruppen-Anomalien (Outlier Detection)

Erkennt Services oder Hosts, die sich von ihrer Peer-Gruppe abheben:

```
spl
```

```
| mstats avg(response_time) AS rt WHERE index=metrics
  groupby host span=5m
| stats mean(rt) AS group_mean stdev(rt) AS group_std by _time
| eval z = (rt - group_mean) / group_std
| where abs(z) > 2.5
```

2. Machine Learning Toolkit (MLTK)

Das MLTK muss als App aus Splunkbase installiert werden. Es bringt eine eigene Python-Sandbox (DLTK) mit.

fit / **apply** - Workflow

Das grundlegende Schema für alle MLTK-Algorithmen:

```
spl

# Modell trainieren:
| mstats avg(cpu) avg(memory) avg(latency) WHERE index=metrics span=1m
| fit <Algorithmus> cpu memory latency into my_model

# Modell anwenden:
| mstats avg(cpu) avg(memory) avg(latency) WHERE index=metrics span=1m
| apply my_model
```

Verfügbare Algorithmen für Metriken im MLTK

DensityFunction

Schätzt die Wahrscheinlichkeitsdichte der Daten und markiert Bereiche niedriger Dichte als Anomalien. Gut für nicht-normalverteilte Metriken:

```
spl

| fit DensityFunction cpu_usage
  threshold=0.01      # Untere 1% der Dichte = Anomalie
  into cpu_density_model
| apply cpu_density_model
| where IsAnomaly = 1
```

DBSCAN (Clustering-basiert)

Findet Cluster normaler Betriebszustände; Punkte außerhalb = Anomalien. Multivariat:

```
spl
```

```
| fit DBSCAN cpu memory latency
  k=2          # Mindestpunkte im Cluster
  eps=0.5      # Clusterradius
| where cluster = -1 # -1 = kein Cluster = Anomalie
```

RandomForestRegressor (supervised)

Für den Fall, dass historische Anomalie-Labels vorliegen:

```
spl

| fit RandomForestRegressor latency
  from cpu memory throughput error_rate
  into latency_model
| apply latency_model
| eval residual = abs(latency - predicted_latency)
| where residual > 50
```

KMeans - Cluster-Anomalieerkennung

Normaler Betrieb = bekannte Cluster; neue Zustände = Anomalie:

```
spl

| fit KMeans cpu memory latency k=3 into ops_clusters
| apply ops_clusters
| eval distance_to_centroid = ...
| where distance_to_centroid > threshold
```

OneClassSVM

Trainiert auf Normaldaten, erkennt alles Abweichende:

```
spl

| fit OneClassSVM cpu memory latency
  nu=0.05          # Erwartete Anomalierate (5%)
  into svm_model
| apply svm_model
| where predicted_anomaly = -1
```

3. MLTK + **predict** kombiniert (Hybrid-Ansatz)

Der in der Praxis robusteste Ansatz kombiniert saisonale Vorhersage mit ML-basierter Residualanalyse:

spl

```
| mstats avg(latency_ms) AS latency WHERE index=metrics span=5m
    earliest=-30d latest=now

| predict latency
    algorithm=LLP
    period=288
    upper95=upper
    lower95=lower

| eval in_band = if(latency >= lower AND latency <= upper, 1, 0)
| eval residual = latency - predicted_latency

| streamstats window=12 stdev(residual) AS residual_std

| eval residual_z = abs(residual / residual_std)

| eval anomaly_score =
    case(
        in_band = 0 AND residual_z > 3, "kritisch",
        in_band = 0 AND residual_z > 2, "warnung",
        true(), "normal"
    )

| where anomaly_score != "normal"
```

Was hier passiert: **predict** fängt saisonale Muster ab, **streamstats** + Residual-Z-Score erkennt persistente Abweichungen vom Vorhersageband – zwei unabhängige Mechanismen müssen gleichzeitig anschlagen.

4. Splunk ITSI (IT Service Intelligence)

Falls ITSI im Einsatz ist, kommen zusätzliche Methoden dazu:

Adaptive Thresholding – Schwellenwerte passen sich automatisch an historische Baselines an, getrennt nach Tageszeit und Wochentag.

Predictive Analytics – ITSI trainiert intern saisonale Modelle pro KPI und berechnet einen **Health Score** (0-100), der den Gesamtzustand eines Services beschreibt, nicht einzelne Metriken.

Event Correlation – Anomalien in verwandten KPIs werden automatisch zu einer Episode gruppiert statt als Einzelalarme zu erscheinen.

5. Anomalieerkennung bei Logdaten

Logdaten unterscheiden sich fundamental von Metriken: statt numerischer Zeitreihen liegen unstrukturierte oder semi-strukturierte Texte vor. Die Anomalieerkennung muss daher zunächst Struktur extrahieren, bevor statistische oder ML-Methoden greifen können.

5.1 Volumen-basierte Anomalien (Log-Rate)

Der einfachste Ansatz: nicht der Inhalt, sondern die **Häufigkeit** von Logeinträgen wird als Metrik behandelt und wie eine normale Zeitreihe ausgewertet.

```
spl
| index=app_logs earliest=-7d
| timechart span=5m count AS log_volume
| predict log_volume algorithm=LLP period=288 upper95=upper lower95=lower
| eval anomaly = if(log_volume > upper OR log_volume < lower, 1, 0)
| where anomaly = 1
```

Typische Anomalien: plötzlicher Anstieg (Fehler-Flood), plötzlicher Einbruch (Service ausgefallen, Logging-Pipeline unterbrochen).

5.2 Fehlerrate-Anomalien (Log-Level-Verhältnis)

Das Verhältnis von ERROR/WARN zu INFO-Einträgen ist ein robusteres Signal als absolute Volumina:

```
spl
| index=app_logs earliest=-24h
| timechart span=5m
  count(eval(level="ERROR")) AS errors
  count(eval(level="INFO")) AS infos
| eval error_ratio = errors / (infos + errors + 0.001)
| streamstats window=60 mean(error_ratio) AS mean_ratio
  stdev(error_ratio) AS std_ratio
| eval z_score = (error_ratio - mean_ratio) / (std_ratio + 0.0001)
| where z_score > 3
| fields _time error_ratio z_score
```

5.3 `anomalydetection` auf aggregierten Logdaten

Der native `anomalydetection`-Befehl lässt sich auf gezählte Log-Events anwenden:

```
spl

| index=app_logs earliest=-7d
| eval hour = strftime(_time, "%H")
| stats count by host, hour, level
| anomalydetection count by host level
| where anomaly = 1
| sort -anomalyScore
```

Erkennt Hosts oder Log-Level-Kombinationen, deren Häufigkeit statistisch ungewöhnlich ist.

5.4 `rare`-Befehl - seltene Muster

Der `rare`-Befehl findet Feldwert-Kombinationen, die statistisch ungewöhnlich selten auftreten - sehr nützlich für neue Fehlermuster oder unbekannte Zugriffsmuster:

```
spl

| index=app_logs earliest=-24h
| rex field=_raw "(?<error_code>ERR-\d+)"
| rare limit=10 error_code host
```

```
spl

| index=access_logs
| rare limit=20 uri_path user_agent
| where percent < 0.01 # Weniger als 0,01% aller Requests
```

5.5 Grok-Parsing + `anomalydetection` (strukturierte Felder)

Erst Logzeilen strukturieren, dann auf extrahierten Feldern detektieren:

```
spl

| index=nginx_logs
| rex field=_raw "\"(?<method>\w+)\s(?<uri>\S+).*?\s(?<status>\d{3})\s(?<bytes>\d+)"
| eval is_error = if(status >= 500, 1, 0)
| timechart span=1m sum(is_error) AS error_count avg(bytes) AS avg_bytes
| anomalydetection error_count avg_bytes
| where anomaly = 1
```

5.6 MLTK: `anomalydetection` auf Text-Features (Term Frequency)

Das MLTK erlaubt es, Log-Nachrichten als **Term-Frequency-Vektoren** zu behandeln und darauf Anomalieerkennung anzuwenden. Neue, unbekannte Fehlermuster werden so erkannt, auch wenn sie noch nie zuvor aufgetreten sind.

```
spl

# Schritt 1: Featurisierung (Trainingsphase auf Normaldaten)
| index=app_logs earliest=-30d latest=-1d level=INFO
| eval message_clean = lower(replace(_raw, "\d+", "N"))
| fit TFIDF message_clean into log_tfidf_model

# Schritt 2: Anomalie-Scoring auf neuen Logs
| index=app_logs earliest=-1h
| eval message_clean = lower(replace(_raw, "\d+", "N"))
| apply log_tfidf_model
| fit OneClassSVM tf_* nu=0.02 into log_ocsvm_model
| apply log_ocsvm_model
| where predicted_anomaly = -1
| table _time host level _raw
```

Der Preprocessing-Schritt `replace(_raw, "\d+", "N")` normalisiert Zahlen (Timestamps, IDs, IPs), damit gleiche Fehlermuster mit verschiedenen Werten als ähnlich erkannt werden.

5.7 MLTK: `LogCluster` - automatische Log-Mustererkennung

`LogCluster` ist ein MLTK-Algorithmus speziell für Logdaten. Er erkennt automatisch wiederkehrende Log-Templates und meldet Einträge, die keinem bekannten Template entsprechen:

```
spl

# Trainieren auf historischen Normaldaten
| index=app_logs earliest=-14d latest=-1d
| fit LogCluster _raw threshold=0.7 into log_cluster_model

# Neue Logs klassifizieren
| index=app_logs earliest=-1h
| apply log_cluster_model
| where cluster_label = "anomaly" # Keinem Template zugeordnet
| table _time host _raw cluster_distance
| sort -cluster_distance
```

Der Parameter `threshold` (0-1) steuert die Empfindlichkeit: je niedriger, desto mehr Einträge werden als Anomalie klassifiziert.

5.8 Sequenz-Anomalien - Ereignisreihenfolge

Manchmal ist nicht ein einzelnes Log-Event anomal, sondern die **Reihenfolge von Ereignissen**.
Beispiel: Ein Login-Erfolg ohne vorherigen Login-Versuch.

```
spl
| index=auth_logs action IN ("login_attempt", "login_success", "logout")
| transaction user maxspan=10m
| eval event_sequence = mvjoin(mvmap(action, action), "->")
| stats count by event_sequence
| sort -count
| where count < 5    # Seltene Sequenzen
```

```
spl
# Erkennung von Login-Erfolg ohne vorherigen Versuch (Anomalie)
| index=auth_logs action IN ("login_attempt", "login_success")
| transaction user maxspan=5m startswith="login_attempt" endswith="login_success"
| where isnull(login_attempt)    # Kein Versuch vor Erfolg
```

5.9 `tstats` + Baseline-Vergleich - schnelle Volumenerkennung

Für sehr große Log-Volumina ist `tstats` deutlich performanter als eine Suche auf Rohdaten:

```
spl
| tstats count WHERE index=app_logs by _time span=5m host
| eventstats avg(count) AS baseline_avg stdev(count) AS baseline_std by host
| eval z = (count - baseline_avg) / (baseline_std + 1)
| where z > 3 OR z < -3
| eval direction = if(z > 0, "spike", "drop")
| table _time host count baseline_avg z direction
```

5.10 Splunk AI Assistant / MLTK Showcase: Log Anomaly Detection App

Splunk bietet über die **MLTK Showcase App** (Splunkbase) fertige Notebooks für Log-Anomalieerkennung auf Basis von:

- **Isolation Forest** auf extrahierten numerischen Log-Features

- **LSTM-Autoencoder** für sequenzbasierte Anomalieerkennung (DLTK erforderlich)
- **Word2Vec + Clustering** für semantische Log-Mustererkennung

Der LSTM-Autoencoder-Ansatz ist der leistungsfähigste, aber auch der aufwändigste – er erfordert die Deep Learning Toolkit (DLTK)-Erweiterung und GPU-Unterstützung für akzeptable Trainingszeiten.

Entscheidungshilfe: Welche Methode wann?

Metriken

Situation	Empfehlung
Schneller Einstieg, univariat	<code>anomalydetection</code> oder <code>streamstats</code> -Z-Score
Saisonale Metriken (Tages-/Wochenmuster)	<code>predict</code> mit LLP + Konfidenzband
Mehrere Metriken korreliert bewerten	MLTK <code>DensityFunction</code> oder <code>DBSCAN</code>
Peer-Gruppen-Vergleich (viele Hosts)	<code>eventstats</code> Z-Score gruppenweise
Maximale Erklärbarkeit	Mahalanobis via MLTK <code>fit</code>
Produktionsreife Pipeline	Hybrid: <code>predict</code> + Residual-Z-Score

Logdaten

Situation	Empfehlung
Log-Volumen plötzlich hoch/niedrig	<code>predict</code> auf <code>timechart count</code>
Fehlerrate-Anstieg	<code>streamstats</code> -Z-Score auf <code>error_ratio</code>
Seltene Fehlercodes / URI-Muster	<code>rare</code> -Befehl
Unbekannte Fehlermuster (Text)	MLTK <code>LogCluster</code> oder TF-IDF + OneClassSVM
Anomale Ereignisreihenfolgen	<code>transaction</code> + Sequenzanalyse
Sehr große Log-Volumina	<code>tstats</code> + Baseline-Vergleich
Maximale Erkennungstiefe	DLTK LSTM-Autoencoder